

Informe de avance

Agente de sugerencias estilísticas para la revista *Innovación y Software*

Especificación, diseño y planificación del producto

Producto

Agente de IA integrado con Open Journal Systems 3.1.2.1 que revisa el estilo de los manuscritos enviados a Innosoft y propone correcciones que el editor valida antes de comunicarlas al autor.

Metodología

Scrum · Spec-Driven Development · priorización RICE

Estado general

Especificación y diseño completos; implementación del *walking skeleton* en curso.

37

specs de historia redactadas

8

specs aprobadas para implementar

44

reglas editoriales formalizadas

6

diagramas de arquitectura y diseño

Contenido

1	Resumen del avance	3
2	Producto	3
2.1	Objetivo	3
2.2	Alcance	4
2.3	Actores	4
2.4	Principios de diseño	4
3	Metodología de trabajo	4
4	Arquitectura y diseño	5
4.1	Contexto del sistema	5
4.2	Despliegue	5
4.3	Modelo de clases	7
4.4	Flujo principal	8
4.5	Estados y datos	9
4.6	API del agente	9
5	Especificaciones	10
5.1	Especificaciones transversales	10
5.2	Decisiones tomadas en la revisión	10
5.3	Índice de especificaciones de historia	10
5.4	Especificaciones aprobadas	12
6	Reglas editoriales	15
6.1	Formato de una regla	15
6.2	Formato del hallazgo	15
7	Plan de desarrollo	16
7.1	Alcance del MVP	16
7.2	Iteración en curso: walking skeleton	17

1. Resumen del avance

El proyecto completó la fase de especificación y diseño. El producto quedó definido de punta a punta: requisitos, reglas editoriales de la revista, arquitectura, contratos entre componentes y una especificación por cada historia de usuario. Las especificaciones de la primera iteración de construcción están aprobadas y su implementación está en curso.

Componente	Contenido	Estado
Catálogo de requisitos	36 historias en 7 épicas, User Story Map con <i>walking skeleton</i> , 12 flujos alternativos, 7 requisitos no funcionales y 44 reglas editoriales	Completo
Priorización	Puntuación RICE de las 36 historias, releases R1–R3 y plan por iteraciones	Completo
Diagramas	Contexto, despliegue, clases, secuencia del flujo principal, estados del manuscrito y modelo de datos	Completo
Spec general (00-producto.md)	Visión, arquitectura, estados, API, datos, RNF y flujos de excepción	Aprobada
Formato de reglas (formato-reglas.md)	Contrato YAML, JSON Schema validado, marcadores, formato del hallazgo y catálogo de evaluadores	Aprobada
Specs de historia (US-00...US-36)	37 specs con propósito, entradas, salidas, reglas, excepciones, criterios de aceptación y plan de pruebas	6 aprobadas · 31 en borrador
Plan de la iteración en curso	6 historias y 30 tareas asignadas	En curso

2. Producto

2.1 Objetivo

Desarrollar un agente de inteligencia artificial integrado con Open Journal Systems que analice los manuscritos enviados a Innosoft en formato PDF, identifique incumplimientos respecto de las normas editoriales de la revista y genere sugerencias estilísticas claras, trazables y revisables por un editor.

2.2 Alcance

Incluye	No incluye
- Manuscritos en español de las secciones Artículo corto, Artículo original y Artículo de revisión. - Corrección de estilo en la evaluación editorial inicial, antes de la revisión por pares. - Entrada en PDF; reglas RE-01... RE-66 priorizadas por release. - Revisión humana obligatoria y publicación como discusión en OJS.	- Revisión del contenido científico, originalidad o plagio. - Corrección ortográfica y gramatical general. - OCR de PDFs escaneados (rechazo controlado). - Manuscritos en inglés y secciones no mapeadas.

2.3 Actores

Actor	Tipo	Responsabilidad
Autor	Humano	Envía el manuscrito y recibe el comentario aprobado en OJS.
Editor	Humano	Revisa, acepta, rechaza o edita sugerencias y las publica.
Administrador	Humano	Gestiona reglas, credenciales y registros.
Plugin OJS	Sistema	Detecta el envío, entrega PDF y metadatos y publica el comentario.
Agente	Sistema	Extrae, valida, genera sugerencias y expone la cola de revisión.
Gemini API	Externo	Redacta sugerencias a partir de hallazgos anonimizados.

2.4 Principios de diseño

- Humano en el ciclo:** ninguna sugerencia llega al autor sin la acción explícita de un editor (RNF-01).
- El motor decide, el LLM redacta:** qué es un incumplimiento lo determinan reglas determinísticas; Gemini solo redacta.
- Trazabilidad por IDs:** regla → hallazgo → sugerencia → decisión → comentario (RNF-04).
- Política fuera del código:** umbrales y mensajes en YAML versionado (RNF-05).
- Fallar por partes:** un error de una etapa no pierde el manuscrito ni bloquea al autor (RNF-06).
- Mínima exposición de datos:** a Gemini solo sale un fragmento anonimizado (RNF-02, Ley N.º 29733).

3. Metodología de trabajo

El desarrollo combina tres prácticas. **Scrum** organiza el trabajo en iteraciones de dos semanas con un equipo de dos investigadores-desarrolladores. **RICE** (Reach × Impact × Confidence / Effort) ordena el backlog y define el *walking skeleton*. **Spec-Driven Development** exige que cada historia tenga su especificación aprobada antes de escribir código.

Especificar → Revisar → Planificar → Implementar → Verificar → Cerrar

Ciclo SDD aplicado a cada historia

Jerarquía de especificaciones. `00-producto.md` fija lo común a todo el sistema y prevalece sobre las demás; `formato-reglas.md` define el contrato de las reglas editoriales; cada `US-nn.md` describe una historia. Todas usan la misma plantilla: propósito, alcance, entradas, salidas, reglas de comportamiento, excepciones, criterios de aceptación en formato Gherkin, plan de pruebas, trazabilidad y estado del ciclo SDD.

Definition of Done.

1. La spec existe, fue revisada por el otro desarrollador y coincide con lo implementado.
2. Pruebas Pytest en verde en GitHub Actions.
3. Código integrado en `main` mediante pull request revisado.
4. Cada hallazgo devuelto tiene un `rule_id` trazable a `rules/*.yaml`.
5. Demo reproducible con `docker compose up`.

4. Arquitectura y diseño

4.1 Contexto del sistema

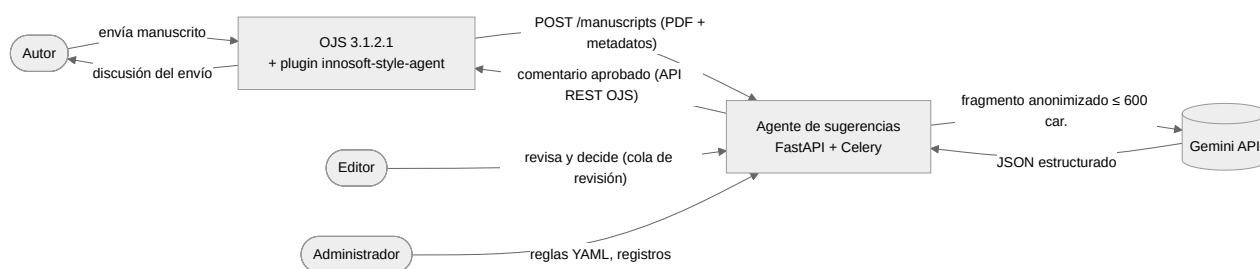


Figura 1: Diagrama de contexto: el agente se sitúa entre OJS, el editor y Gemini.

4.2 Despliegue

El sistema se despliega en dos servidores: el de la revista (OJS 3.1.2.1 con el plugin `innosoft-style-agent`) y el del agente (contenedores Docker con la API FastAPI y PostgreSQL). La arquitectura objetivo añade un *worker* Celery con Redis para el procesamiento asíncrono; en la primera etapa ese procesamiento se ejecuta con `BackgroundTasks` de FastAPI sin cambiar los contratos (decisión D-04).

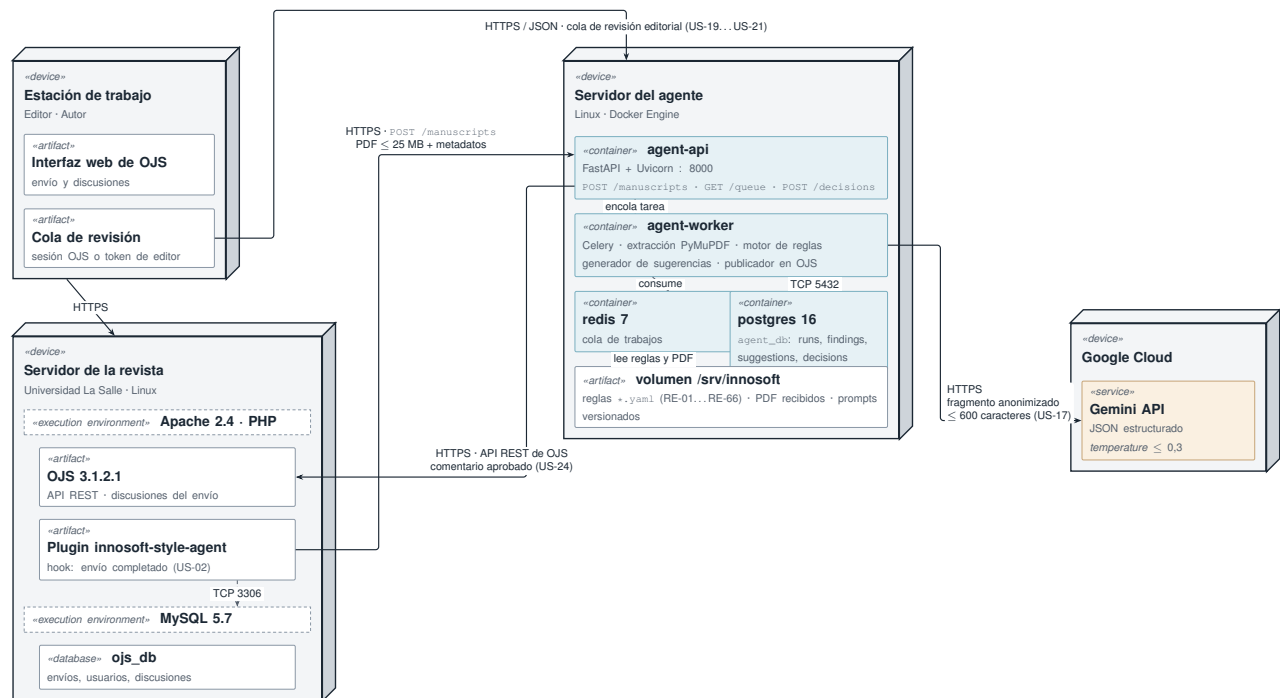


Figura 2: Diagrama de despliegue.

4.3 Modelo de clases

Seis paquetes: `api`, `extraction`, `model.document`, `rules`, `suggestion` y `review & publishing`. Las interfaces `PdfExtractor`, `LlmClient` y `OjsGateway` permiten sustituir PyMuPDF, Gemini y OJS por dobles de prueba.

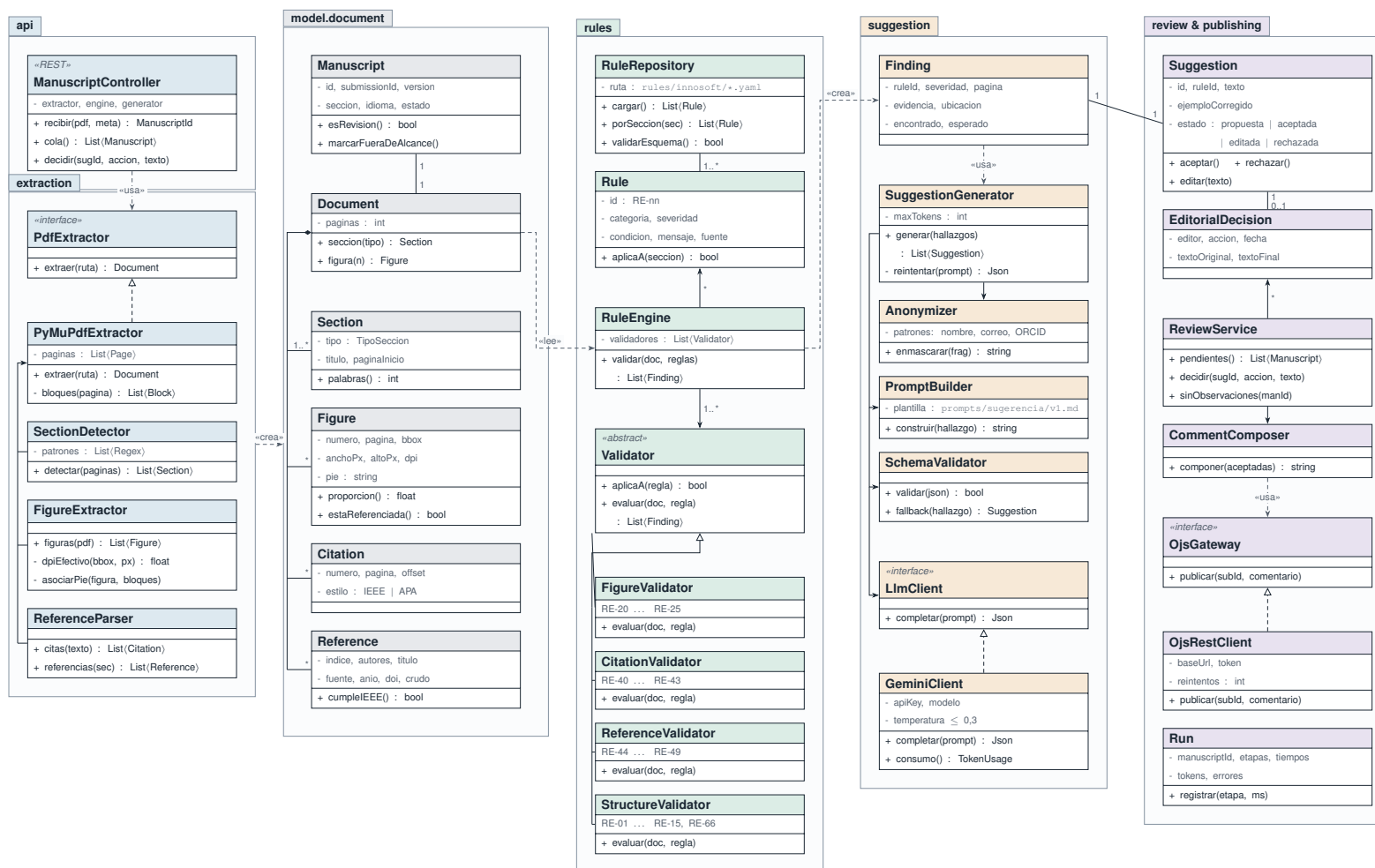


Figura 3: Diagrama de clases.

4.4 Flujo principal

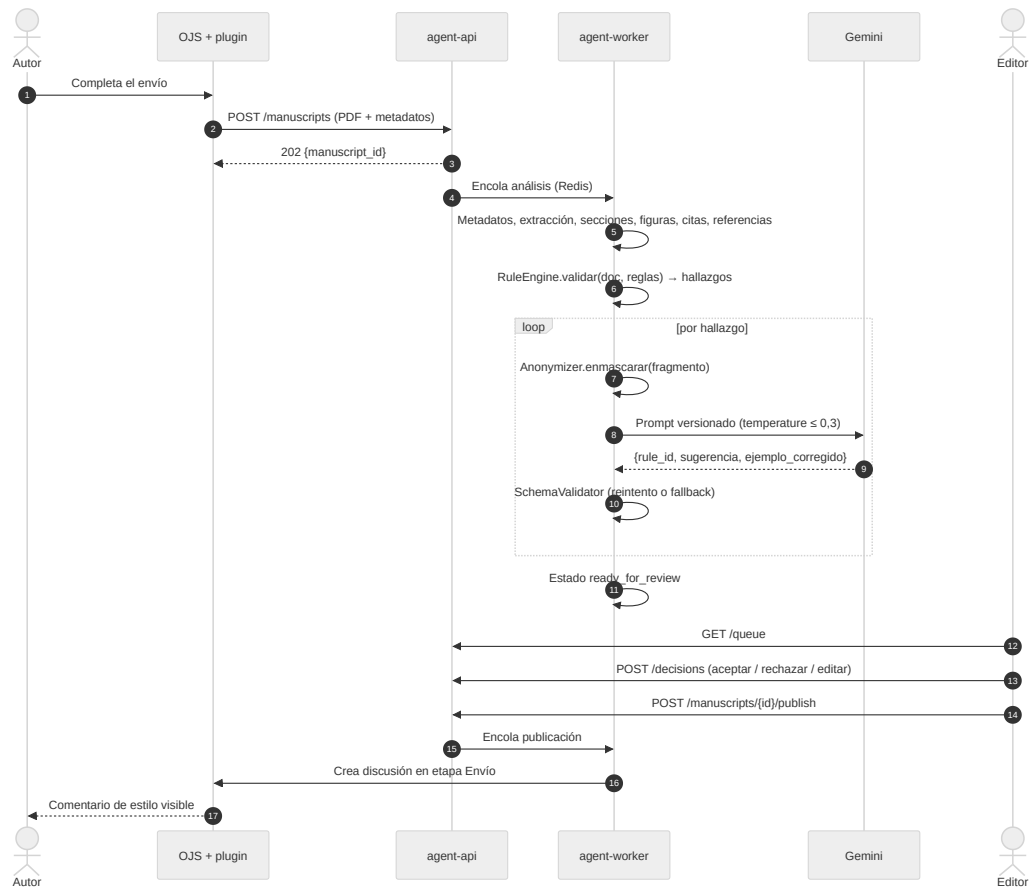


Figura 4: Secuencia del flujo principal, desde el envío del autor hasta el comentario en OJS.

4.5 Estados y datos

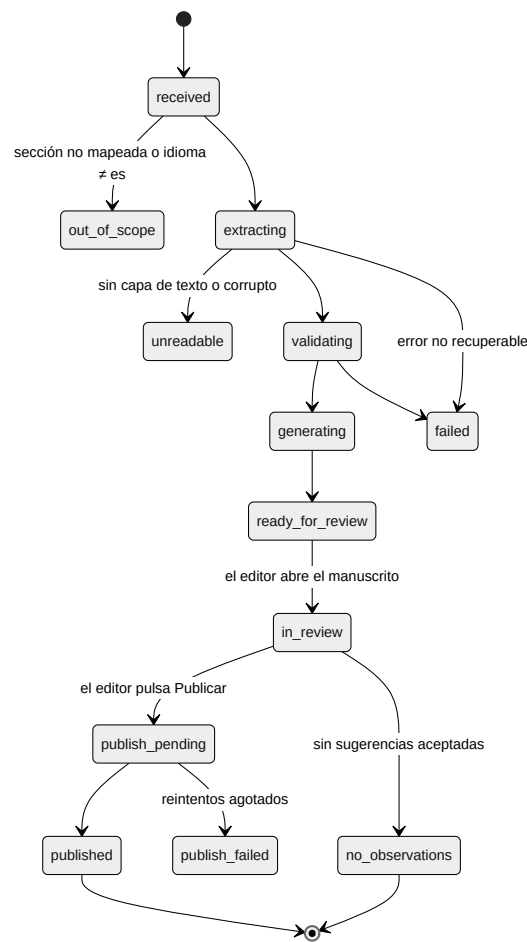


Figura 5: Ciclo de vida del manuscrito.

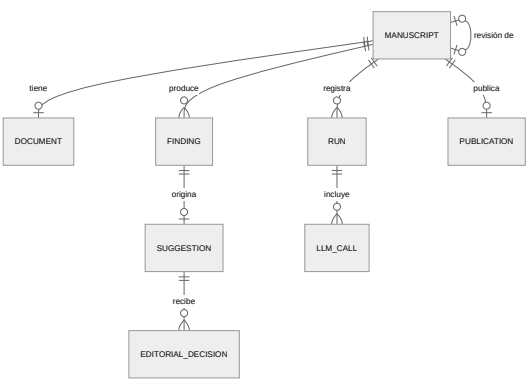


Figura 6: Modelo de datos (esbozo).

4.6 API del agente

Endpoint	Rol	Historia
POST /manuscripts	Plugin OJS	US-01
GET /manuscripts/{id}/findings	Editor	US-10
GET /manuscripts/{id}/suggestions	Editor	US-20
POST /manuscripts/{id}/publish	Editor	US-24
GET /queue	Editor	US-19
POST /decisions	Editor	US-21
GET /rules?section=	Editor / administrador	US-09
POST /admin/rules/reload	Administrador	US-30
GET /runs?manuscript_id=	Administrador	US-28
GET /health	Monitoreo	US-00

5. Especificaciones

5.1 Especificaciones transversales

Documento	Contenido	Estado
00-producto.md	Objetivo y criterios de éxito, alcance, actores, contexto, despliegue, módulos, flujo principal, estados del manuscrito y de la sugerencia, contratos y API, modelo de datos, principios, RNF-01...07, ALT-01...12, stack, estructura del repositorio, proceso y glosario.	Aprobada
formato-reglas.md	Organización de <code>rules/innosoft/*.yaml</code> , campos de una regla, severidad, tipo de detección, JSON Schema (validado contra ejemplos reales), marcadores del mensaje, formato del hallazgo, catálogo de evaluadores de las 44 reglas, versión del conjunto de reglas y pruebas obligatorias.	Aprobada
_template.md	Plantilla para nuevas especificaciones.	Vigente

5.2 Decisiones tomadas en la revisión

ID	Decisión	Specs afectadas
D-01	Un reenvío idéntico (mismo <code>submission_id</code> y <code>sha256</code>) devuelve el manuscrito existente con 200 OK; no se duplica ni se re-analiza.	US-01, US-31
D-02	RE-22 (figura dentro de márgenes) sale de US-10 y se evalúa en US-35.	US-10, US-35
D-03	Supuesto aprobado para RE-20: mínimo 531 × 1328 px (alto × ancho) para toda figura raster; la variante panorámica se añadirá al YAML si la revista la precisa.	US-10, formato-reglas
D-04	Procesamiento asíncrono con <code>BackgroundTasks</code> de FastAPI en la primera etapa; Celery + Redis se incorporan después sin cambiar los contratos.	US-00, US-01, 00-producto

5.3 Índice de especificaciones de historia

ID	Historia	Épica	Rel.	Iteración	RICE	Estado
US-00	Especificación general del producto y configuración del entorno	Transversal	R1	2	—	Aprobada
US-01 *	Recibir PDF por API	A · Recepción	R1	2	600	Aprobada
US-02	Plugin OJS: hook al completar un envío	A · Recepción	R1	5	75	Borrador
US-03	Obtener metadatos del envío (sección, título, idioma)	A · Recepción	R1	3	160	Borrador
US-04 *	Extraer texto y páginas	B · Extracción	R1	2	600	Aprobada
US-05 *	Detectar secciones IMRyD [†]	B · Extracción	R1	3	100	Borrador
US-06 *	Extraer figuras: posición, tamaño y pie	B · Extracción	R1	2	216	Aprobada
US-07 *	Extraer citas en texto [n]	B · Extracción	R1	3	320	Borrador
US-08 *	Extraer lista de referencias [†]	B · Extracción	R1	3	160	Borrador
US-09 *	Cargar reglas Innosoft desde YAML	C · Validación	R1	2	600	Aprobada
US-10 *	Validar figuras: tamaño y proporción	C · Validación	R1	2	216	Aprobada
US-11	Validar figuras: pie, numeración y orden [†]	C · Validación	R1	3	112	Borrador
US-12 *	Validar citas IEEE en el texto	C · Validación	R1	3	256	Borrador

ID	Historia	Épica	Rel.	Iteración	RICE	Estado
US-13 *	Validar referencias IEEE (formato, orden y mínimo) [†]	C · Validación	R1	3	80	Borrador
US-14	Validar estructura, resumen y palabras clave	C · Validación	R2	Backlog	30	Borrador
US-15 *	Redactar sugerencia con Gemini	D · Sugerecias	R1	4	160	Borrador
US-16 *	Validar el esquema de respuesta del LLM	D · Sugerecias	R1	4	400	Borrador
US-17	Anonimizar el contexto antes de enviarlo al LLM	D · Sugerecias	R1	4	320	Borrador
US-18	Reintentos, límite de tokens y fallback determinístico [†]	D · Sugerecias	R1	4	96	Borrador
US-19 *	Ver cola de manuscritos pendientes	E · Revisión	R1	4	200	Borrador
US-20 *	Ver sugerencias con su evidencia	E · Revisión	R1	4	160	Borrador
US-21 *	Aceptar, rechazar o editar cada sugerencia	E · Revisión	R1	5	160	Borrador
US-22	Agregar una sugerencia manual	E · Revisión	R2	Backlog	64	Borrador
US-23	Marcar manuscrito como "sin observaciones"	E · Revisión	R2	Backlog	40	Borrador
US-24 *	Publicar comentario en la discusión del envío	F · Publicación	R1	5	160	Borrador
US-25	Reintento y cola si OJS no responde	F · Publicación	R2	Backlog	16	Borrador
US-26	Vincular el comentario a la versión del manuscrito	F · Publicación	R2	Backlog	50	Borrador
US-27	Notificar al editor por correo de OJS	F · Publicación	R2	Backlog	80	Borrador
US-28 *	Registro de ejecución por manuscrito	G · Administración	R1	4	200	Borrador
US-29	Registro de consumo y latencia de Gemini	G · Administración	R2	Backlog	200	Borrador
US-30	Editar reglas sin redespigie	G · Administración	R2	Backlog	40	Borrador
US-31	Detectar reenvío (versión 2 del manuscrito)	A · Recepción	R2	Backlog	25	Borrador
US-32	Panel de métricas	G · Administración	R3	Backlog	26,7	Borrador
US-33	Auditoría de decisiones editoriales	G · Administración	R3	Backlog	50	Borrador
US-34	Detectar tablas y sus títulos	B · Extracción	R3	Backlog	15	Borrador
US-35	Detectar figuras recortadas o fuera de margen	B · Extracción	R3	Backlog	20	Borrador
US-36	Validar formato de página (fuente, márgenes, numeración)	C · Validación	R3	Backlog	6,7	Borrador

* Historia del *walking skeleton*. † Alcance acotado para el MVP (ver sección 7).

5.4 Especificaciones aprobadas

US-00 · Especificación general del producto y configuración del entorno

Aprobada

Como equipo de desarrollo, *quiero* una especificación general aprobada y un entorno reproducible, *para* que todas las historias posteriores se especifiquen, construyan y prueben sobre la misma base.

Historia técnica que habilita el ciclo Spec-Driven Development: fija la visión del producto en `specs/00-producto.md`, el formato de reglas en `specs/formato-reglas.md`, la plantilla de spec y el entorno (repositorio, CI, Docker Compose, datos de prueba) sobre el que se implementa el walking skeleton.

Criterios de aceptación clave

- Dado el repositorio clonado en una máquina limpia con Docker, cuando se ejecuta `docker compose up`, entonces la API responde 200 en `GET /health` y la base de datos acepta conexiones.
- Dado un pull request con una prueba fallida, cuando se ejecuta GitHub Actions, entonces el check de CI queda en rojo y el merge está bloqueado.
- Dado `specs/00-producto.md`, cuando ambos developers la revisan, entonces queda en estado *Aprobada* con los RNF-01..07 y ALT-01..12 enumerados.
- Dado la carpeta `tests/fixtures/pdfs/`, cuando se inspecciona, entonces contiene los 5 PDFs descritos y ninguno incluye correos u ORCID reales.

Pruebas: 4 casos · **Decisiones:** D-04

US-01 · Recibir PDF por API

Aprobada

Como plugin OJS, *quiero* enviar un PDF y sus metadatos a un endpoint del agente, *para* iniciar el análisis.

Es el punto de entrada de todo el flujo: recibe el manuscrito, lo valida mínimamente, lo persiste y devuelve un identificador para que el procesamiento continúe de forma asíncrona sin bloquear a OJS.

Criterios de aceptación clave

- Dado un PDF válido de 2 MB y metadatos con `submission_id`, cuando el plugin hace `POST /manuscripts`, entonces responde 202 con un `manuscript_id` UUID y el registro queda con estado `received`.
- Dado un archivo `.docx` renombrado a `.pdf`, cuando se envía al endpoint, entonces responde 400 y no se crea registro ni archivo.
- Dado un PDF de 26 MB, cuando se envía al endpoint, entonces responde 413 y no se crea registro ni archivo.
- Dado un envío aceptado, cuando se consulta `storage/{manuscript_id}.pdf`, entonces el archivo existe y su `sha256` coincide con el registrado.
- Dado un PDF ya recibido con el mismo `submission_id`, cuando el plugin lo envía de nuevo, entonces responde 200 con el mismo `manuscript_id`, `duplicate: true`, y no se crea un segundo registro ni un nuevo análisis.

Pruebas: 7 casos · **Decisiones:** D-01, D-04

US-04 · Extraer texto y páginas**Aprobada**

Como agente, *quiero* el texto por página con coordenadas, *para* ubicar cada hallazgo.

Produce la representación base del documento (`Document`) sobre la que trabajan todas las demás extracciones y reglas. Sin texto no hay reglas.

Criterios de aceptación clave

- Dado un PDF de texto de 15 páginas, cuando se ejecuta la extracción, entonces el `Document` tiene 15 páginas con bloques, `bbox`, fuente y tamaño y el estado es `extracted`.
- Dado el PDF escaneado del corpus, cuando se ejecuta la extracción, entonces el estado es `unreadable` con motivo `no_text_layer` y el editor ve el aviso.
- Dado un PDF de 30 páginas, cuando se mide la extracción, entonces termina en menos de 10 s.
- Dado una extracción exitosa, cuando se consulta la base de datos, entonces el `Document` está persistido como JSONB y asociado al `manuscript_id`.

Pruebas: 4 casos

US-09 · Cargar reglas Innosoft desde YAML**Aprobada**

Como administrador, *quiero* que las reglas editoriales vivan en archivos YAML versionados y validados, *para* ajustar umbrales y mensajes sin tocar el código.

Separa la política editorial (qué se exige) del código (cómo se evalúa). Habilita el motor completo, la trazabilidad `rule_id` y RNF-05.

Criterios de aceptación clave

- Dado `figuras.yaml` con RE-20..RE-27 válidas, cuando arranca la API, entonces las 8 reglas quedan cargadas y `GET /rules?section=original` las devuelve.
- Dado una regla sin `id`, cuando arranca la API, entonces el arranque se detiene con un mensaje que indica archivo y posición.
- Dado una regla con `seccion_aplicable: [corto]`, cuando se consulta `GET /rules?section=original`, entonces esa regla no aparece.

Reglas: RE-20, RE-21, RE-22, RE-23, RE-24, RE-25, RE-26, RE-27 · **Pruebas:** 5 casos

US-06 · Extraer figuras: posición, tamaño y pie**Aprobada**

Como agente, *quiero* conocer, para cada imagen, su página, `bbox`, dimensiones en píxeles, DPI efectivo y pie de figura asociado, *para* validar las reglas de figuras.

Las figuras mal dimensionadas, pequeñas o mal etiquetadas son el principal problema de estilo de la revista. Esta historia produce los datos objetivos sobre los que operan RE-20..RE-25.

Criterios de aceptación clave

- Dado un PDF con 3 figuras raster con pie, cuando se extraen figuras, entonces hay 3 elementos con página, `bbox`, píxeles, `dpi` y pie `Figura n..`
- Dado una figura de 1200 px de ancho impresa a 6 pulgadas, cuando se calcula el DPI, entonces `dpi_x` es 200 (± 1).
- Dado un PDF generado desde LaTeX con figura vectorial, cuando se extraen figuras, entonces la figura aparece con `kind=vector` y sin DPI.
- Dado el PDF sin figuras del corpus, cuando se extraen figuras, entonces `figures=[]` y no hay errores.

Reglas: RE-20, RE-21, RE-23 · **Pruebas:** 5 casos

US-10 · Validar figuras: tamaño y proporción**Aprobada**

Como editor, *quiero* que el agente detecte figuras demasiado pequeñas, de baja resolución o fuera de márgenes, *para* pedir al autor figuras legibles antes de la revisión por pares.

Primera regla de valor real para el editor y cierre del walking skeleton: convierte los datos de US-06 en hallazgos trazables a RE-20 y RE-21.

Criterios de aceptación clave

- Dado una figura de 300 × 600 px, cuando se ejecuta el motor, entonces se genera un hallazgo RE-20 con `valor_encontrado="300 × 600 px"`.
- Dado una figura de 800 × 2000 px a 300 dpi, cuando se ejecuta el motor, entonces no se genera hallazgo RE-20 ni RE-21.
- Dado una figura raster a 150 dpi, cuando se ejecuta el motor, entonces se genera un hallazgo RE-21 con severidad alta.
- Dado un evaluador que lanza una excepción, cuando se ejecuta el motor, entonces las demás reglas producen sus hallazgos y el run registra `rule_error`.
- Dado un manuscrito procesado, cuando se llama `GET /manuscripts/{id}/findings`, entonces se devuelven los hallazgos con `rule_id` existente en `rules/*.yaml`.

Reglas: RE-20, RE-21 · **Pruebas:** 6 casos · **Decisiones:** D-02, D-03

6. Reglas editoriales

Las normas de Innosoft se formalizaron en 44 reglas con identificador, severidad, fuente (directrices para autores, plantilla Word y plantilla LaTeX) y tipo de detección: determinística (D), con apoyo del LLM (L) o informativa (I). Cada regla tiene asignado un evaluador y parámetros iniciales.

Categoría	Reglas	D	L o D/L	I o L/I	Severidad alta
Estructura y metadatos	16	10	5	1	6
Figuras y tablas	12	10	0	2	3
Citas y referencias IEEE	10	9	1	0	6
Formato de página	6	5	1	0	0
Total	44	34	7	3	15

6.1 Formato de una regla

Las reglas viven en archivos YAML versionados y se validan contra un JSON Schema al arrancar el sistema; cambiar un umbral no requiere tocar código.

```
- id: RE-20
  categoria: figuras
  descripcion: "Imagen de al menos 531 x 1328 px (alto x ancho)"
  condicion:
    evaluador: figure_min_pixels
    parametros: {min_height_px: 531, min_width_px: 1328}
  severidad: alta
  detectable: D
  mensaje: "La {figura} (pagina {pagina}) mide {encontrado}. La revista solicita un minimo de {esperado}."
  seccion_aplicable: [corto, original, revision]
  fuente: {documentos: [a], url: "https://revistas.ulasalle.edu.pe/innosoft/about/submissions"}
```

6.2 Formato del hallazgo

Todo evaluador devuelve hallazgos con la misma estructura, que conserva el identificador de la regla hasta el comentario final:

```
{"rule_id": "RE-20", "severidad": "alta", "pagina": 5,
  "ubicacion": {"tipo": "figura", "numero": 3, "bbox": [90, 200, 520, 430]},
  "valor_encontrado": "300 x 600 px", "valor_esperado": ">= 531 x 1328 px"}
```

7. Plan de desarrollo

Iteración	Meta	Historias	Estado
1	Especificación y diseño	Catálogo, priorización RICE, diagramas, 00-producto, formato-reglas y specs de las 37 historias	Completado
2	Walking skeleton	US-00, US-01, US-04, US-09, US-06, US-10	En curso
3	Procesamiento documental y reglas de citas y referencias	US-03, US-05, US-07, US-08, US-11, US-12, US-13 + prueba técnica de la API de OJS	Planificado
4	Integración de Gemini e interfaz editorial	US-17, US-15, US-16, US-18, US-28, US-19, US-20	Planificado
5	Revisión editorial e integración con OJS	US-21, US-24, US-02	Planificado
6	Evaluación y versión final	Métricas (precisión, recall, F1, tasa de aceptación), ajustes y manuales	Planificado

7.1 Alcance del MVP

Para concentrar el MVP en los problemas prioritarios de la revista (figuras y referencias IEEE), cinco historias se implementan con un alcance acotado; el resto de su alcance queda en el backlog de la release R2.

ID	Historia	Alcance en el MVP
US-05	Detectar secciones IMRyD	Solo Resumen, Palabras clave y Referencias; el diccionario completo de variantes pasa a R2.
US-08	Extraer lista de referencias	Segmentación y numeración de entradas; el parseo por campos pasa a R2.
US-11	Validar figuras: pie, numeración y orden	RE-23 y RE-24; RE-25 (mención de la figura en el texto) pasa a R2.
US-13	Validar referencias IEEE (formato, orden y mínimo)	RE-44, RE-46, RE-47 y RE-49; RE-45 (patrón IEEE por tipo) y RE-48 pasan a R2.
US-18	Reintentos, límite de tokens y fallback determinístico	Reintentos con backoff y fallback; el presupuesto de tokens por manuscrito pasa a R2.

Las historias US-14, US-22, US-23, US-25, US-27, US-29 y las de la release R3 permanecen especificadas en borrador en el backlog para desarrollos posteriores.

7.2 Iteración en curso: walking skeleton

Meta: contar con una versión mínima en Docker que reciba un PDF de Innosoft, extraiga texto y figuras, aplique RE-20 y RE-21 desde `rules/innosoft/figuras.yaml` y devuelva un JSON de incumplimientos con ubicación.

ID	Entregable	Spec
US-00	Repositorio con integración continua, Docker Compose (API y PostgreSQL) y corpus de 5 PDFs de prueba anonimizados	Aprobada
US-01	POST <code>/manuscripts</code> con validación, almacenamiento e idempotencia	Aprobada
US-04	Extracción de texto por página con coordenadas y detección de PDFs ilegibles	Aprobada
US-09	Carga y validación de reglas YAML; GET <code>/rules</code>	Aprobada
US-06	Extracción de figuras: bbox, píxeles, DPI efectivo, pie y figuras vectoriales	Aprobada
US-10	Motor de reglas y evaluadores RE-20 y RE-21; GET <code>/manuscripts/{id}/findings</code>	Aprobada